

Anatomy of a CI/CD Pipeline



What Is a CI/CD Pipeline?

CI – Continuous Integration



- Developers regularly merge code changes into a shared repository.
- Each change is automatically built and tested.
- Helps catch issues early before deployment.

CD – Continuous Delivery or Continuous Deployment



- Automatically takes tested code and prepares it for release.
- With Continuous Delivery, releases are approved manually.
- With Continuous Deployment, releases go to production automatically.

Why Pipelines Exist



Faster and more consistent delivery



Catch errors early



Reduce manual work



Improve quality and security



Enable reliable and repeatable deployments

Manual Deployment (Without a Pipeline)



- ✗ Many manual steps
- ✗ Prone to human error
- ✗ Inconsistent results
- ✗ Slower releases
- ✗ Harder to roll back
- ✗ Higher risk in production

Automated Pipeline (With CI/CD)



- ✓ Automatic build and test
- ✓ Consistent and repeatable
- ✓ Faster and safer releases
- ✓ Early detection of issues
- ✓ Easy rollback
- ✓ More reliable production deployments

Simple CI/CD Pipeline Flow



Code

Developer commits code to the repository



Build

Pipeline builds the application



Test

Automated tests run to verify quality



Deploy

Application is deployed to the target environment



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

1. FULL PIPELINE FLOW



2. WHAT EACH STAGE DOES

Developer	Writes code and pushes to the repository.
Git	Stores the code and triggers the pipeline on events like push or pull request.
CI	Runs the defined pipeline jobs on a runner.
Build	Compiles the code, installs dependencies and creates a deployable package.
Test	Runs unit, integration and other automated tests.
Security	Scans for vulnerabilities, secrets and policy violations.
Artifact	Produces a versioned artifact (e.g. container image).
Deploy	Deploys the artifact to the target environment (dev, staging or production).
Verify	Runs health checks and confirms the application is working correctly.

3. PIPELINE AS QUALITY GATES



4. BIGGER PICTURE



Key Takeaway:

A CI/CD pipeline automates the journey from code to a working application with built-in quality, security and verification gates.



1. WHAT IS A PIPELINE TRIGGER?

A pipeline trigger is an event that tells your CI/CD system to start running the pipeline. When the trigger happens, the CI platform detects the event, creates a pipeline run and executes the defined jobs on a runner.



IMPORTANT

Not every Git event should deploy to production. Most events should run build, test and security checks first. Deployments to production usually require extra controls.

2. COMMON PIPELINE TRIGGERS

Pushes



Triggered when you push code to a branch (e.g. main or develop).

Common use:
Run build, test and security checks.

Pull Requests



Triggered when a pull request is created or updated.

Common use:
Run checks before merging the code.

Merges



Triggered when a pull request is merged into a target branch (e.g. main).

Common use:
Build, test and often deploy to non-production environments.

Tags



Triggered when a Git tag is created (e.g. v1.0.0).

Common use:
Build and deploy a versioned release.

Manual Triggers



Triggered manually by an engineer from the CI platform UI.

Common use:
Deploy to staging or production with approval.

Scheduled Pipelines



Triggered at specific times (e.g. every night or every week).

Common use:
Run tests, security scans or build updates (e.g. dependencies).

3. HOW A TRIGGER STARTS THE PIPELINE



1. Git Event

Push, PR, merge, tag, manual or scheduled event happens.



2. Repository

The CI platform detects the event in the repository.



3. CI/CD Platform

Creates a new pipeline run based on your pipeline configuration file.



4. Runner

Picks up the job and starts executing the pipeline steps.



5. Pipeline Jobs

Runs build, test, security, deploy and other jobs as defined.

4. WHY NOT EVERY GIT EVENT SHOULD DEPLOY TO PRODUCTION?



- A simple code push may be experimental or incomplete.
- Pull requests can contain untested or risky changes.
- Frequent deployments increase the chance of production issues.
- Some changes require manual review and approval.
- You may want different environments for different branches (e.g. develop → staging, main → production).
- Security and compliance checks must pass before production.

5. BEST PRACTICES

- ✓ Use branch-based triggers.
- ✓ Run tests and security scans on every push.
- ✓ Deploy to staging before production.
- ✓ Use manual approval for production.
- ✓ Use tags for official releases.
- ✓ Use scheduled pipelines for routine checks.
- ✓ Keep your pipeline configuration in version control.



Key Takeaway: Triggers start the pipeline, but you control what happens next. Use the right trigger for the right purpose.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

1. WHAT EXECUTES PIPELINE JOBS?

A CI runner is the actual machine or environment that executes the pipeline jobs defined in your pipeline file. It checks out your code, installs dependencies, runs commands and performs all the steps you define.



IMPORTANT

- A pipeline definition file (e.g. YAML) does not execute itself.
- It is just a set of instructions.
- A CI platform (e.g. GitHub Actions, GitLab CI, Jenkins) uses a runner to execute those instructions.

2. HOSTED vs SELF-HOSTED RUNNERS

Hosted Runners



- ✓ Provided and managed by the CI platform (e.g. GitHub, GitLab, Azure DevOps).
- ✓ Ready to use.
- ✓ Common operating systems preinstalled (e.g. Linux, Windows, macOS).
- ✓ No infrastructure management.
- ✓ Good for most projects and beginners.

Self-Hosted Runners



- ✓ You provision and manage the runner yourself.
- ✓ Can run on your own server, VM or cloud instance.
- ✓ Useful for private networks, custom tools or specific security requirements.
- ✓ You control the environment and configurations.
- ✓ Requires maintenance, updates and monitoring.

3. KEY PARTS OF THE RUNNER ENVIRONMENT

Workspace



- A fresh directory where your repository is checked out.
- Contains your code and temporary files.
- Usually cleaned after the job completes.

Environment



- Operating system (e.g. Ubuntu, Windows).
- Preinstalled tools (e.g. Git, Docker).
- Environment variables.
- Network access (e.g. internet, internal resources).

Permissions



- Defines what the runner is allowed to do.
- Access to repositories, registries and cloud services.
- Uses tokens or credentials.
- Should follow the principle of least privilege.

Job Execution



- The runner executes each job step in order.
- Runs the commands you defined in the pipeline file.
- Sends logs and results back to the CI platform.

4. WHY A PIPELINE FILE DOES NOT EXECUTE ITSELF



- The pipeline file is a configuration file.
- It only defines what should happen (e.g. build, test, deploy).
- A CI platform needs a runner (a real machine or container) to execute those steps.
- Without a runner, nothing will run.



Think of it this way:

- The pipeline file is the recipe.
- The runner is the kitchen.
- The CI platform is the chef that finds the kitchen and runs the recipe.

5. FLOW: FROM GIT EVENT TO JOB EXECUTION

1. Git Event



You push code, open a pull request, merge, create a tag or trigger manually.

2. CI Platform



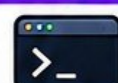
Detects the event, reads the pipeline file and creates a pipeline run.

3. Runner



A runner is assigned to execute the jobs in the pipeline.

4. Job Execution



The runner checks out the code and runs the defined steps (build, test, deploy, etc.).



Key Takeaway:

A CI runner is the engine that executes your pipeline. Without a runner, the pipeline file is just instructions.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

1. WHAT HAPPENS IN THIS STAGE?

In this stage, the runner gets your code, prepares a clean working directory, installs the required dependencies and sets up the environments so the next jobs (build, test, etc.) can run.

2. REPOSITORY CHECKOUT



- The runner clones or checks out your repository from Git.
- It downloads the code for the specific branch, commit or tag that triggered the pipeline.
- This gives the runner a local copy of your project in its workspace.

3. BRANCH AND COMMIT BEING BUILT

- The pipeline checks out the exact branch, commit or tag that triggered it.
- This ensures you test and deploy the same code that was pushed or merged.
- You can see the branch and commit details in the pipeline logs.



WHY IT MATTERS

- Ensures the pipeline uses the exact code and dependencies it should.
- Gives a clean and consistent environment.
- Helps avoid "works on my machine" problems.
- Makes builds and tests reproducible.

Example (GitHub Actions)

```

- name: Checkout code
  uses: actions/checkout@v4
  with:
    fetch-depth: 0
  
```

4. DEPENDENCY INSTALLATION



- Installs the required dependencies for your project (e.g. npm, pip, Maven).
- Uses a clean environment, so you know exactly what is installed.
- If a dependency fails to install, the pipeline stops.

Example (Node.js)

```

- name: Install dependencies
  run: npm ci
  
```

5. ENVIRONMENT PREPARATION



- Sets up the tools and runtime required for your project (e.g. Node.js, Python, Java, Docker).
- Configures environment variables.
- Sets up any services or containers needed for the job (e.g. databases for tests).

Example (Set up runtime)

```

- name: Set up Node.js
  uses: actions/setup-node@v4
  with:
    node-version: '20'
  
```

6. CACHING



- Caching saves dependencies and build files so they don't need to be downloaded or built again on every run.
- This makes pipelines faster and reduces external network dependency.

Example (Cache dependencies)

```

- name: Cache npm packages
  uses: actions/cache@v4
  with:
    path: ~/.npm
    key: npm-${runner.os}-${hashFiles('**/package-lock.json')}
  
```

7. WHY REPRODUCIBLE ENVIRONMENTS MATTER



- Every pipeline run should produce the same result when using the same code and configuration.
- Prevents unexpected bugs caused by different tool versions or missing dependencies.
- Helps you detect real issues early.
- Makes it easier to debug and collaborate.
- Essential for reliable deployments to staging and production.



Key Takeaway

- The checkout and preparation stage gives the runner a clean, consistent and reproducible environment.
- It ensures the rest of the pipeline runs with the correct code, tools and dependencies.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



1. WHAT "BUILD" MEANS



- The build stage takes your source code, processes it and turns it into a deployable artifact.
- It can compile code, install dependencies, run build scripts and create a package or container image.
- The output of this stage is something that can be deployed to an environment.

2. COMPILATION (WHERE APPLICABLE)



Compile

- Some languages need to be compiled before they can run (e.g. Java, Go, C#, C++).
- Compilation converts your source code into executable files or binaries.

Example: Compile a Java application
`mvn clean package`

3. DEPENDENCY RESOLUTION



- The build stage downloads and installs all required dependencies for your project (e.g. npm, pip, Maven).
- It uses a dependency file (e.g. package.json, requirements.txt, pom.xml) to get the exact versions.
- This ensures the application builds with the right libraries and tools.

Example: Install dependencies (Node.js)

```
# Install project dependencies
npm ci
```

Example: Install dependencies (Python)

```
# Install project dependencies
pip install -r requirements.txt
```

4. APPLICATION PACKAGING



- After compilation and dependency installation, the applications are packaged.
- The package can be a JAR file, WAR file, ZIP file, or any other format depending on the technology.
- This packaged artifact is what gets deployed in later stages.

Example: Build and package (Maven)

```
# Build and package the application
mvn clean package
```

5. DOCKER IMAGE BUILDS



- Many modern applications are packaged as Docker images.
- The build stage uses a Dockerfile to create an image that contains the application and everything it needs to run.
- The image is usually pushed to a container registry (e.g. Docker Hub, ECR, ACR).

Example: Build a Docker image

```
- Build Docker image
docker build -t myapp:1.0 .

# Tag and push to registry
docker tag myapp:1.0 myregistry/myapp:1.0
docker push myregistry/myapp:1.0
```

6. BUILD FAILURES



- Builds can fail due to compilation errors, missing dependencies, test failures, or misconfigured build scripts.
- When a build fails, the pipeline stops and later stages do not run.
- You should check the build logs to find the first meaningful error and fix the issue.

Common reasons for build failures

- Syntax or compilation errors
- Missing or incompatible dependencies
- Incorrect build configuration
- Insufficient permissions
- Resource or disk space issues

7. WHY THE SAME COMMIT SHOULD PRODUCE PREDICTABLE RESULTS



- The same commit should always produce the same build output, no matter where or when it runs.
- Reproducible builds reduce "works on my machine" problems.
- They make it easier to debug, test and deploy.
- Use versioned dependencies, locked dependency files (e.g. package-lock.json, requirements.txt), and consistent build environments (e.g. Docker or official CI images).



Key Takeaway

The build stage converts source code into a consistent, deployable artifact. A successful build should be repeatable, reliable and produce the same result for the same commit in any environment.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



1. WHAT IS AUTOMATED TESTING?



- Automated testing runs predefined tests on your code inside the CI pipeline.
- It checks if the application works as expected.
- It helps catch bugs early before deployment.



WHY IT MATTERS

- Improves code quality.
- Catches issues early.
- Reduces broken deployments.
- Builds confidence in your releases.
- Saves time and manual effort.

2. UNIT TESTS



- Test small parts of the code (functions, methods).
- Run quickly.
- Check individual components in isolation.

```
# Run unit tests (example)
npm test
```

3. INTEGRATION TESTS



- Test how different parts of the application work together (e.g. app + database).
- Slower than unit tests.
- Helps find issues in real interactions.

```
# Run integration tests (example)
npm run test:integration
```

4. SMOKE TESTS



- Basic tests to check if the main features work after a build or deployment.
- Run quickly.
- Often used after deployment to a test environment.

```
# Run smoke tests (example)
npm run test:smoke
```

5. EXIT CODES



- Tests return an exit code to indicate success or failure.
- Exit code 0 = success
- Exit code 1 (or other non-zero code) = failure

Exit Code	Meaning
0	Success (tests passed)
1	Failure (tests failed)
>1	Other errors

6. TEST REPORTS



- Test reports show detailed results of what passed or failed.
- They help you understand which tests failed and why.
- Most CI platforms can display test reports in the pipeline UI.

```
# Example: Generate test report with Jest
npm test -- --reporters=default \
--reporters=jest-junit
```

7. WHY FAILED TESTS SHOULD STOP THE PIPELINE



- Failed tests usually mean the code has a bug or is not ready.
- Deploying broken code can cause issues in later environments.
- Stopping the pipeline early saves time, cost and prevents bigger problems.

8. FLOW: BUILD → TEST → PASS OR FAIL



Build
Application is built



Run Tests

Unit, integration and smoke tests are executed



Tests Pass
(Exit code 0)
Pipeline continues to the next stage (e.g. deploy)



Tests Fail
(Non-zero exit code)
Pipeline stops and later stages do not run

9. EXAMPLE CI PIPELINE (GITHUB ACTIONS)

```
name: Build and Test
on:
  push:
    branches: [ main ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Install dependencies
        run: npm ci
      - name: Run tests
        run: npm test
```



Key Takeaway: Automated tests give you confidence that your code works. If the tests fail, the pipeline should stop immediately.



1. WHAT ARE QUALITY GATES?



- Quality gates are automated checks that must pass before the pipeline can move to the next stage.
- They enforce code quality, security, and project standards.
- If a gate fails, the pipeline stops.



WHY IT MATTERS

- Prevents low-quality code from reaching deployment.
- Catches issues early when they are cheaper to fix.
- Keeps your codebase clean, secure and maintainable.
- Reduces production risks.
- Enforces team and organizational standards.

2. LINTING



- Checks your code for potential errors, bad practices and style issues.
- Helps keep the code consistent and readable.
- Common tools: ESLint (JavaScript), Pylint (Python), Checkstyle (Java).

```
# Example (Python - flake8)
flake8 .
```

3. FORMATTING CHECKS



- Ensures the code follows a standard format.
- Reduces unnecessary differences in pull requests.
- Common tools: Prettier (JavaScript), Black (Python), Spotless (Java).

```
# Example (Prettier)
npx prettier --check .
```

4. STATIC ANALYSIS



- Analyzes code without running it to find bugs, security issues and code smells.
- Helps improve maintainability and security.
- Common tools: SonarQube, CodeQL, Bandit (Python), Semgrep.

```
# Example (SonarScanner)
sonar-scanner
```

5. TEST THRESHOLDS



- Ensures a minimum test coverage or quality threshold.
- Prevents merging code with insufficient test coverage.
- Example: Require at least 80% code coverage.

```
# Example (Jest with coverage threshold)
npx jest --coverage --coverageThreshold={"global":{"lines":80}}
```

6. POLICY CHECKS



- Enforces organization rules and compliance requirements.
- Checks for allowed licenses, secrets, dependencies and configuration policies.
- Common tools: Open Policy Agent (OPA), Conftest, Checkov, Trivy (for policy and security checks).

```
# Example (Checkov - infrastructure policy check)
checkov -d .
```

7. THE PIPELINE WITH QUALITY GATES



Code
Commit



Linting
Pass



Formatting
Pass



Static Analysis
Pass



Tests
Pass



Policy Checks
Pass



Proceed
to Next Stage
(e.g. Deploy)

8. WHY "THE CODE BUILDS" DOES NOT MEAN "THE CODE IS READY"



- The code might compile or build, but still have bugs, security issues, poor quality or fail organizational standards.
- Without quality gates, broken or risky code can reach later stages and even production.
- Quality gates ensure the code is truly ready, not just buildable.



Key Takeaway

- Quality gates catch issues early.
- They enforce standards and improve code quality.
- A passing build is just the start — the code must also pass all quality gates before moving forward.

1. WHY SECURITY CHECKS?



- Security checks find vulnerabilities and risky configurations before deployment.
- They prevent security issues from reaching production.
- They protect your application, data, users and infrastructure.
- They are a critical part of a reliable pipeline, not optional.



IMPORTANT

- Security should be integrated early in the pipeline (shift left).
- Automated security checks catch common issues quickly.
- They reduce risk, cost and manual effort.
- They help you meet compliance and organizational security standards.

2. DEPENDENCY SCANNING



- Scans third-party libraries and dependencies for known vulnerabilities.
- Helps you identify and fix vulnerable packages early.
- Common tools: Snyk, Dependabot, OWASP Dependency-Check (or built-in CI tools).

```
# Example (OWASP Dependency-Check)
dependency-check --scan .
--format HTML
```

3. SECRET SCANNING



- Checks your code, commits and files for exposed secrets (e.g. API keys, passwords, cloud credentials).
- Prevents accidental leaks.
- Common tools: GitLeaks, TruffleHog, GitHub secret scanning.

```
# Example (GitLeaks)
gitleaks detect \
--source .
```

4. STATIC APPLICATION SECURITY TESTING (SAST)



- Analyzes your source code for security vulnerabilities without running the application.
- Finds issues like injection flaws, insecure configurations, etc.
- Common tools: SonarQube, Semgrep, CodeQL.

```
# Example (Semgrep)
semgrep scan --config=auto .
```

5. CONTAINER IMAGE SCANNING



- Scans container images for vulnerabilities in OS packages and application dependencies.
- Helps you catch issues before pushing to a registry or deploying.
- Common tools: Trivy, Gype, Anchore, Docker Scout.

```
# Example (Trivy)
trivy image myapp:1.0.0
```

6. INFRASTRUCTURE AS CODE SCANNING



- Scans infrastructure code (e.g. Terraform, CloudFormation, Kubernetes manifests) for security misconfigurations.
- Helps you follow security best practices and prevent risky resources.
- Common tools: Checkov, tfsec, Terrascan.

```
# Example (Checkov)
checkov --directory .
```

7. WHY SECURITY BEFORE DEPLOYMENT?



- Stops vulnerable code and configurations from reaching production.
- Reduces the risk of breaches, data loss and downtime.
- It is cheaper and faster to fix issues early.
- Helps maintain compliance and builds customer trust.

8. WHAT SHOULD HAPPEN WHEN A CRITICAL ISSUE IS DISCOVERED?



- The pipeline should fail immediately.
- Deployment to later stages should be blocked.
- The issue should be clearly reported in the pipeline logs and test reports.
- The development team should be notified.
- The issue must be fixed and the pipeline re-run.
- No workarounds or overrides for critical security issues.



Key Takeaway

- Security checks are essential quality gates in the pipeline.
- They catch real risks before deployment.
- A critical security issue should always stop the pipeline until it is fixed.

1. WHAT IS AN ARTIFACT?



- An artifact is the output produced by the pipeline after the build and test stages.
- It is a versioned, immutable file or image that can be deployed.
- Artifacts can be application packages, container images, or other build outputs.



WHY IT MATTERS

- Gives you a consistent, tested output.
- Can be deployed to different environments (staging, production).
- Reduces "works on my machine" problems.
- Makes rollbacks easier.
- Improves security and traceability.
- Allows the same artifact to be promoted across environments.

2. APPLICATION PACKAGES



- Compiled or packaged application code.
- Examples: JAR (Java), WAR, ZIP, TAR, RPM, DEB, .NET packages, Python wheels.
- Stored in an artifact repository for later use.

```
# Example (Maven - JAR package)
mvn package
# Output: target/myapp-1.0.0.jar
```

3. CONTAINER IMAGES



- A complete, runnable image that contains the application and its dependencies.
- Built using a Dockerfile.
- Stored in a container registry (e.g. Docker Hub, ECR, GCR, Azure Container Registry).

```
# Example (Docker build)
docker build -t myapp:1.0.0 .
docker push myapp:1.0.0
```

4. BUILD OUTPUTS



- Other files produced during the build process.
- Examples: compiled binaries, static assets (HTML, CSS, JS), configuration files, database migration files.
- These outputs are also stored as artifacts.

```
# Example (Build output)
ls -l build/
# Output: binaries, assets, configs
```

5. ARTIFACT REPOSITORIES



- A central storage for build artifacts (packages, binaries, etc.).
- Examples: Nexus, Artifactory, GitHub Packages, AWS CodeArtifact.
- Helps manage, secure and distribute artifacts across teams and environments.

```
# Example (Upload artifact - Nexus)
mvn deploy
# Uploads package to Nexus repository
```

6. CONTAINER REGISTRIES



- A dedicated storage for container images.
- Examples: Docker Hub, Amazon ECR, Google GCR, Azure ACR, GitHub Container Registry.
- Stores versioned images that can be pulled and deployed later.

```
# Example (Push to Amazon ECR)
docker tag myapp:1.0.0 123456789.dkr.ecr
.amazonaws.com/myapp:1.0.0
docker push 123456789.dkr.ecr.amazonaws.com/myapp:1.0.0
```

7. VERSIONING



- Artifacts are versioned so you can track exactly what was built and deployed.
- Common versioning methods: semantic versioning (e.g. 1.0.0), commit SHA, build number or tags.
- Helps with traceability, rollbacks and reproducibility.

```
# Examples
myapp:1.0.0
myapp:latest
myapp:abc123 (commit SHA)
myapp:v1.0.0-20240920 (build number)
```

8. WHY ENGINEERS PROMOTE THE SAME TESTED ARTIFACT



- ✓ Ensures what was tested is exactly what is deployed.
- ✓ Reduces environment inconsistencies.
- ✓ Minimizes risks and unknown changes.
- ✓ Makes rollbacks simple and reliable.
- ✓ Improves security and compliance (traceability).

1. WHAT ARE PIPELINE SECRETS?



- Secrets and credentials are sensitive values used by the pipeline to access external systems.
- They include API tokens, registry credentials, cloud credentials, database passwords and other confidential information.
- They must be stored securely and never hardcoded in pipeline files or repositories.

2. API TOKENS



- Used to authenticate with APIs (e.g. GitHub, GitLab, third-party services).
- Allow the pipeline to perform actions such as creating releases, triggering deployments or accessing resources.

```
# Example (GitHub token from secret)
curl -H "Authorization: token $GITHUB_TOKEN" \
https://api.github.com/user
```

3. REGISTRY CREDENTIALS



- Used to authenticate with container registries (e.g. Docker Hub, Amazon ECR, Google GCR, Azure ACR).
- Required to push and pull container images.
- Stored as secrets in the CI/CD platform or a secret management tool.

```
# Example (Login to Docker registry)
docker login -u $DOCKER_USERNAME \
-p $DOCKER_PASSWORD
```

4. CLOUD CREDENTIALS



- Used to access cloud resources (e.g. AWS, Azure, GCP).
- Allow the pipeline to deploy infrastructure or applications.
- Should have minimal permissions (required only for specific tasks).

```
# Example (Using cloud credentials)
aws s3 ls --region $AWS_REGION
az account show
```

5. SECRET STORES



- Secrets should be stored in secure secret management tools instead of in code.
- Examples: GitHub Secrets, GitLab CI Variables, AWS Secrets Manager, Azure Key Vault, HashiCorp Vault.
- They provide encryption, access control and auditing.

6. ENVIRONMENT VARIABLES



- Used to pass non-sensitive and sensitive values to pipeline jobs.
- Sensitivity settings hide secret values in logs.
- Can be defined at the repository, organization or environment level.

```
# Example (Using a secret environment variable)
echo "Deploying to $ENVIRONMENT"
echo "Using API key: $API_KEY"
```

7. LEAST PRIVILEGE



- Use the minimum permissions required for each credential.
- Limit access to only the resources and actions needed.
- Reduces the impact if a credential is compromised.
- Use separate credentials for different environments (dev, staging, production).

8. WHY CREDENTIALS MUST NEVER BE HARDCODED



- Hardcoded credentials can be exposed in public or internal repositories.
- They can be stolen and used by attackers.
- They are difficult to rotate and control.
- They violate security best practices and compliance requirements.
- Always use secret stores and environment variables provided by your CI/CD platform.

9. GOOD VS BAD EXAMPLE



Bad (Hardcoded)

```
aws_access_key = "AKIA..."
aws_secret_key = "abcd1234"
```



Good (Using Secrets)

```
aws_access_key = $AWS_ACCESS_KEY_ID
aws_secret_key = $AWS_SECRET_ACCESS_KEY
```



Key Takeaway

- Keep all credentials and secrets out of your code.
- Use secure secret stores and environment variables.
- Follow the principle of least privilege.
- Rotate credentials regularly.
- Never expose secrets in logs, commit history or code.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



1. CLEAR DISTINCTION AMONG ALL THREE

CONTINUOUS INTEGRATION (CI)



- Focuses on building and testing code automatically.
- Runs on every change (e.g. push or pull request).
- Identifies issues early.
- Produces a tested artifact.

CONTINUOUS DELIVERY (CD)



- Extends CI by automatically preparing releases for production.
- Deploys to staging or a pre-production environment.
- Requires manual approval to deploy to production.
- Ensures the application is always in a deployable state.

CONTINUOUS DEPLOYMENT (CD)



- Extends Continuous Delivery.
- Automatically deploys to production without manual approval.
- Every change that passes tests is released to users.
- Requires strong automation, monitoring and rollback capabilities.



2. WHERE CI ENDS



- CI ends after the code is built, tested and a versioned artifact is produced.
- It does not deploy to production.
- The output is a tested artifact ready for release.
- Deployment is not part of CI.

3. WHAT CONTINUOUS DELIVERY ADDS



- Automatically deploys the tested artifact to a staging or pre-production environment.
- Adds release automation, environment configuration and deployment scripts.
- Keeps the application always ready for production release.
- Requires manual approval to deploy to production.

4. WHAT CONTINUOUS DEPLOYMENT CHANGES



- Removes manual approval.
- Automatically deploys to production after tests pass.
- Every approved change is released to users.
- Requires high confidence in tests, security checks and monitoring.
- Needs solid rollback and incident response processes.

5. MANUAL APPROVAL VS AUTOMATIC PRODUCTION RELEASE



Continuous Delivery (Manual Approval)

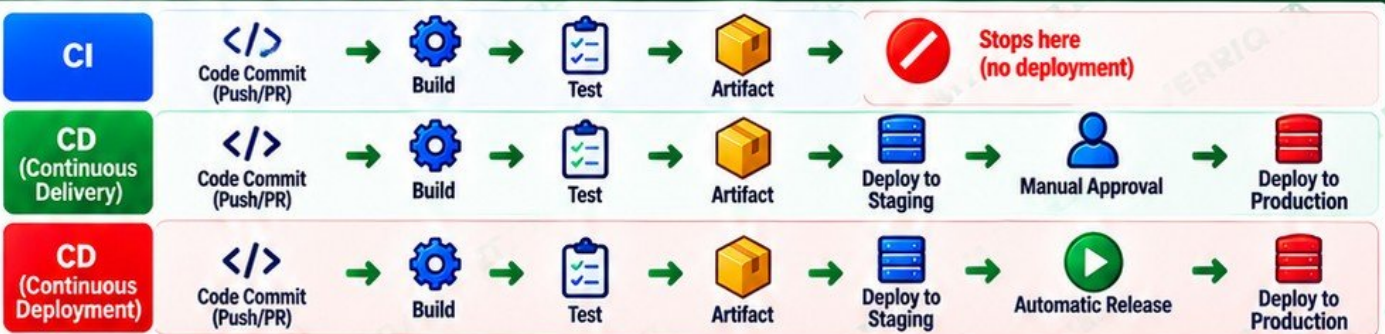
- Code is deployed to staging automatically.
- A human approves the release to production.
- Gives teams control and time for final checks.



Continuous Deployment (Automatic Release)

- No human approval.
- Every change that passes all checks is deployed to production automatically.
- Faster releases but requires strong testing, monitoring and rollback.

6. PIPELINE FLOW COMPARISON



7. KEY DIFFERENCES AT A GLANCE

Feature	Continuous Integration (CI)	Continuous Delivery (CD)	Continuous Deployment (CD)
Build and test code	Yes	Yes	Yes
Creates artifact	Yes	Yes	Yes
Deploys to staging	No	Yes	Yes
Manual approval for production	Not applicable	Yes	No
Automatically deploys to production	No	No	Yes
Goal	Find issues early	Always ready for release	Release every change



8. KEY TAKEAWAY

- CI builds and tests the code.
- Continuous Delivery automates release to staging and requires manual approval for production.
- Continuous Deployment automatically releases to production.
- All three work together in a complete CI/CD pipeline.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



1. DEVELOPMENT



- Used for active development and early testing.
- Frequent changes and deployments.
- May use sample or mocked data.
- Not intended for end users.
- Helps developers test features before moving to test environment.

Example Use Cases:

- Feature development
- Unit and integration testing
- Testing new dependencies

2. TEST



- Used for more formal testing (e.g. integration, system testing, security testing).
- Closer to production configuration than development.
- Helps validate that the application works as expected in a controlled environment.
- May use sanitized or realistic data.

Example Use Cases:

- Integration testing
- Functional testing
- Security and performance testing

3. STAGING



- A pre-production environment.
- Replicates the production environment as closely as possible.
- Used for final testing and validation.
- Helps catch issues that did not appear in lower environments.
- Often used for user acceptance testing (UAT).

Example Use Cases:

- UAT (User Acceptance Testing)
- Final regression testing
- Performance and load testing

4. PRODUCTION



- The live environment used by real users.
- Should be stable, secure and highly monitored.
- Only tested and approved releases are deployed here.
- Changes should be minimal, controlled and reversible.
- Any issue here can impact customers, revenue and reputation.

Example Use Cases:

- Live application for end users
- Processing real data
- High availability and monitoring

5. ENVIRONMENT-SPECIFIC CONFIGURATION

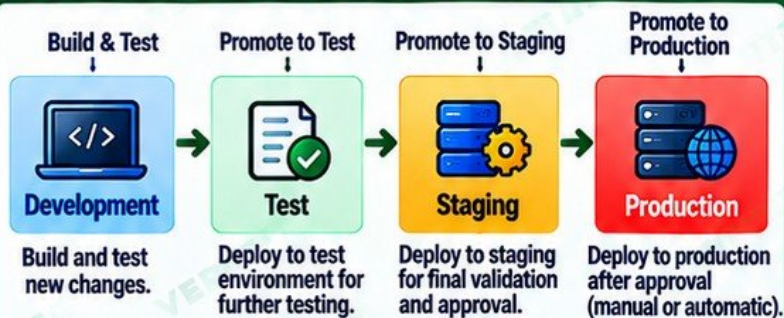


- Each environment may have different configuration settings (e.g. database, API endpoints, feature flags).
- Use environment variables, configuration files or a configuration management tool.
- Keep sensitive data (secrets) out of code and use a secret store.
- Helps the same code run in different environments without changes.

Development Test Staging Production

```
# Example environment-specific configuration
APP_ENV=development
API_URL=https://dev-api.example.com
DATABASE_URL=postgres://dev-db:5432/app
FEATURE_FLAG_X=true
LOG_LEVEL=debug
```

6. PROMOTION BETWEEN ENVIRONMENTS



7. WHY STAGING SHOULD HELP EXPOSE PROBLEMS BEFORE PRODUCTION



- Staging is a safe place to find issues before real users are affected.
- It uses a production-like configuration and data, so you can catch environment-related problems.
- Helps identify performance issues, misconfigurations, missing dependencies and integration problems.
- Reduces the risk of downtime, failed releases and urgent rollbacks in production.
- Improves release confidence and stability.

8. COMPLETE FLOW: FROM DEVELOPMENT TO PRODUCTION



The Deployment Stage

What Happens During Deployment?

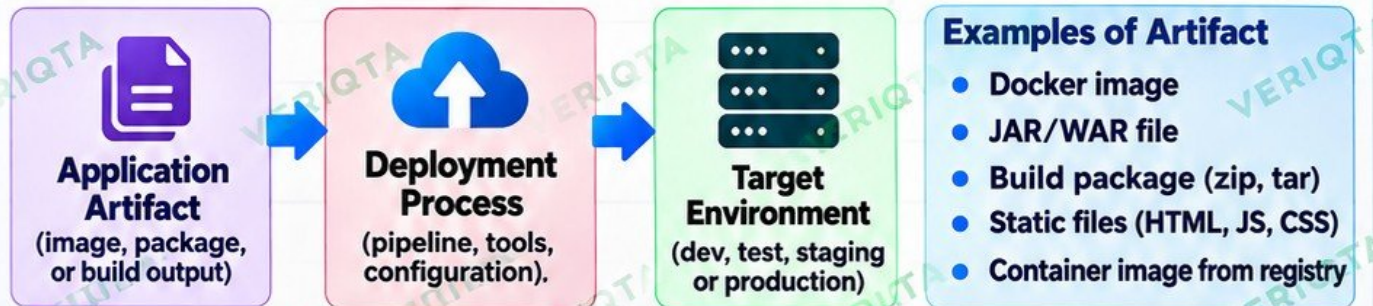
- 1 The pipeline takes the tested and approved artifact.
- 2 Connects to the target environment.
- 3 Applies configuration for that environment.
- 4 Releases the new version (using a deployment strategy).
- 5 Verifies that the deployment completed.



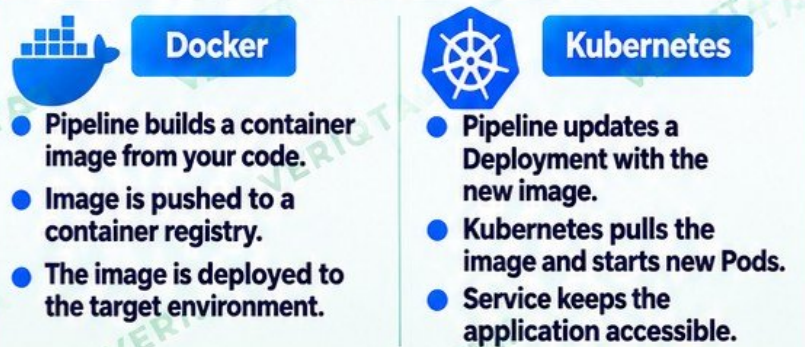
Key Point

Deployment means moving the application artifact to a target environment and making it run. It does not automatically mean the application is working correctly.

Application Artifact → Target Environment



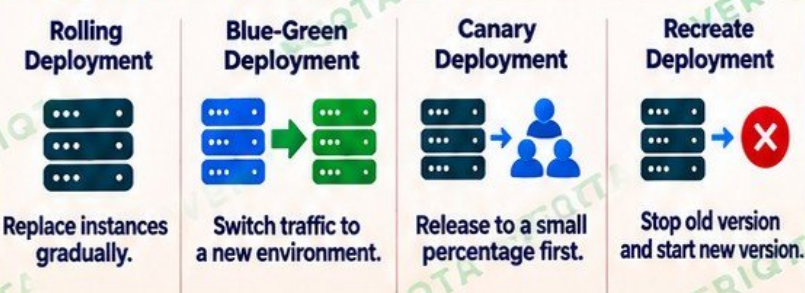
Containers and Kubernetes



Configuration

- 
- Environment-specific configuration (dev, test, prod)
 - Environment variables
 - ConfigMaps and Secrets (in Kubernetes)
 - Application settings (files or external config)
 - Database connection details
 - Feature flags

Deployment Strategies



Important: Deployment Success ≠ Application Success

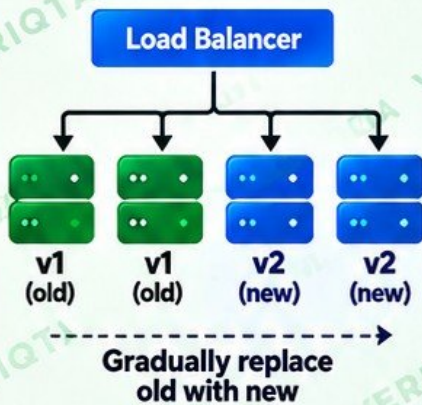
- ✗ Deployment job may show success.
- ✗ Pods or containers may be running.
- ✗ But the application can still be broken (e.g. failed health checks, errors, or bad config).
- ✓ Always verify the application after deployment (health checks, logs, metrics, and real user functionality).

Deployment Strategies

A deployment strategy defines how you release a new version of your application to users. It controls the order, speed and amount of traffic that goes to the new version.

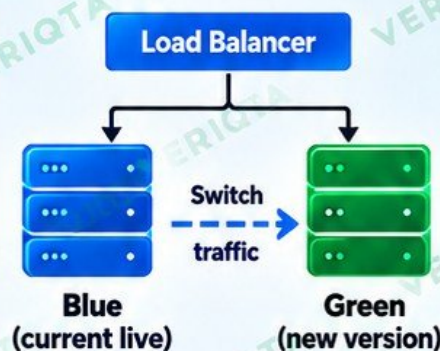
1 Rolling Deployment

- Replaces old instances gradually with new ones.
- Some versions run at the same time during the update.
- Users continue to be served while the deployment happens.



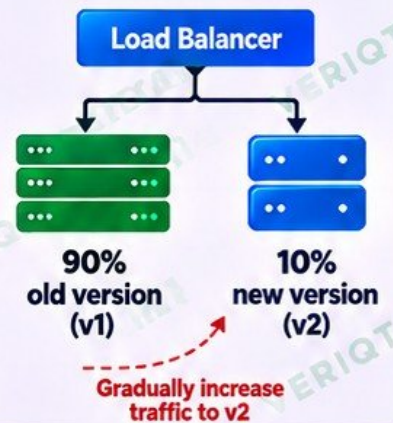
2 Blue-Green Deployment

- Runs two identical environments (Blue and Green).
- Only one environment receives user traffic at a time.
- After testing, traffic is switched to the new environment instantly.



3 Canary Deployment

- Releases the new version to a small percentage of users first.
- Monitors the new version for issues.
- If healthy, gradually increases traffic to the new version.



Tradeoffs at a Junior Engineer Level

Factor	Rolling Deployment	Blue-Green Deployment	Canary Deployment
Downtime	No downtime (gradual update)	No downtime (fast switch)	No downtime (small % first)
Complexity	Simple	Moderate	Moderate
Resource usage	Low	High (runs two environments)	Low to moderate
Risk	Lower than full replace	Low (easy rollback)	Lowest (small user impact)
Rollback	Moderate (rollback steps)	Very easy (switch back)	Quick (reduce traffic)
Best use case	Most common, general use	High availability systems	Large or risky changes

Why Production Systems Avoid Replacing Everything at Once?



- Reduces the risk of a major outage if the new version has issues.
- Allows monitoring and rollback before all users are affected.
- Helps maintain availability and stability.
- Large systems have many dependencies, so small, controlled changes are safer.
- Even tested code can behave differently in production due to real user traffic, data and environment differences.

Post-Deployment Verification

After deployment, you verify that the application is actually working in the target environment. Deployment completed only means the code was released, not that the application is healthy, available or performing correctly.



1 Health Checks



- Check if the application is responding.
- Verify key endpoints (e.g. /health).
- Used by load balancers and Kubernetes probes.

Example (Kubernetes)

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
```

2 Smoke Tests



- Quick functional tests after deployment.
- Verify core features are working.
- Catch major problems early.

Example (simple HTTP check)

```
curl -f https://myapp.com/health
if [ $? -eq 0 ]; then
  echo "Application is healthy"
fi
```

3 Metrics



- Monitor key metrics like:
 - CPU and memory usage
 - Request rate
 - Response time (latency)
 - Error rate (5xx)
 - Throughput

Example (Prometheus query)

```
rate(http_requests_total[5m])
```

4 Logs



- Check application logs for errors.
- Look for unexpected exceptions or warnings.
- Confirm the application started correctly.

Example (Kubernetes)

```
kubectl logs deployment/myapp
--tail=100
```

5 Application Availability



- Verify the application is accessible to users.
- Test from inside and outside the cluster.
- Check DNS, load balancer and service endpoints.

Example

```
curl -I https://myapp.com
# Check HTTP status code
(should be 200)
```

6 Error Rates



- Monitor 4xx and 5xx errors.
- High error rates may indicate issues.
- Compare with normal baseline after deployment.

Example (Prometheus)

```
rate(http_requests_total{
  status=~"5.." }[5m])
```

Why a Green Deployment Job Can Still Hide a Broken Application



Pipeline

Deploy Success

- Application may crash after startup.
- Health check may be too simple.
- Database or external service may fail.
- Only part of the application may work.
- Performance issues may appear under real traffic.
- Users may see errors even though deployment succeeded.

You Should Verify

- Application is running and accessible
- Key features work (smoke tests)
- No critical errors in logs
- Metrics are normal
- Error rates are low
- Real user functionality works
- Monitor for a period after deployment

When the Pipeline Fails

A CI/CD pipeline can fail at different stages. Each failure has a different cause, different error messages and a different fix. Understanding where and why it failed helps you resolve issues faster.

1 Build Failure



- Code or dependency error.
- Build command fails.
- Compilation error (e.g. Java, Node.js, etc).

Example Error (Build)

```
ERROR: Compilation failed
src/App.java:10: error:
cannot find symbol
symbol:   class UserService
```

2 Test Failure



- Unit or integration tests failed.
- A test assertion did not pass.
- Pipeline stops to prevent a bad release.

Example Error (Test)

```
FAIL tests/test_login.py
AssertionError:
Expected status code 200
but got 500
```

3 Authentication Failure



- Invalid or expired credentials.
- Missing permissions.
- Cannot access cloud, registry or cluster.

Example Error (Auth)

```
ERROR: Access denied
The provided credentials
are not valid or have expired.
(HTTP 401)
```

4 Registry Failure



- Cannot push or pull image from registry.
- Network or permission issue.
- Registry is unavailable.

Example Error (Registry)

```
ERROR: failed to push image
403 Forbidden
repository does not exist
or may require 'docker login'
```

5 Deployment Failure



- Deployment command failed.
- Configuration error.
- Cluster or environment issue.

Example Error (Deploy)

```
Error: deployment failed
pods "myapp" not found
due to invalid configuration
or resource limits.
```

6 Health-Check Failure



- Application deployed but not healthy.
- Health endpoint failing.
- Service not responding.

Example Error (Health Check)

```
Health check failed
HTTP 503 Service Unavailable
Readiness probe failed:
connection refused
```

How to Identify Which Stage Failed



1. Check the pipeline summary

- See which stage failed (build, test, deploy, etc).



2. Open the logs for that stage

- Read from the top to find the first meaningful error.



3. Understand the error

- Look at error message, exit code and surrounding logs.



4. Form a hypothesis

- Identify the most likely cause based on the error and recent changes.



5. Verify and fix

- Make the fix, rerun the pipeline and confirm success.



Key Takeaways

- Each stage can fail for different reasons.
- Always read the logs to find the first meaningful error.
- Do not just rerun the pipeline without understanding the cause.
- Fix the root cause, not just the symptom.
- Use clear error messages, exit codes and context to troubleshoot.

> Common Exit Codes

- 0 Success (no error)
- 1 General error
- 2 Misuse of shell command
- 127 Command not found
- 130 Script terminated (e.g. Ctrl+C)
- 137 Killed (e.g. out of memory)
- 255 General failure



Best Practices

- Use clear and descriptive logs.
- Monitor pipeline failures.
- Set up notifications (Slack, Email).
- Fail fast on critical issues.
- Keep secrets and credentials valid.
- Use proper health checks.
- Review recent code changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Rollback and Recovery

Rollback means returning your application to a previous known-good version when a new release causes issues. Recovery is the process of getting the system back to a healthy state.

1 What Rollback Means



- Revert to a previous working version.
- Use a known-good artifact or image.
- Restore application and configuration to a stable state.

2 Previous Known-Good Artifact



v3
(buggy)



v2
(known-good)



v1
(older)

- Keep previous versions in your artifact repository or container registry.
- Tag and version your releases.
- Make sure the previous version is tested and ready to use.

3 Rollback Flow



1. Failed release
(v3 deployed)



2. Detect the issue
(monitoring, alerts, users)



3. Stop the rollout
(if still in progress)



4. Rollback to known-good
version (v2)



5. Verify the application
is healthy

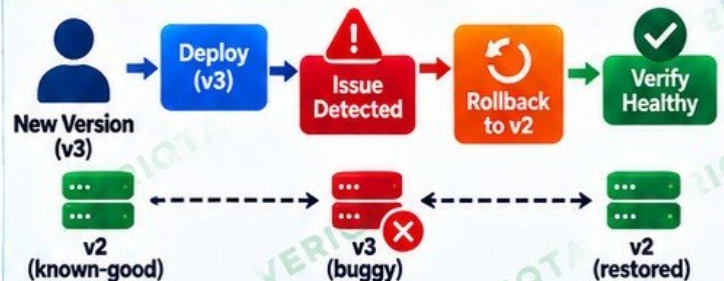
4 Deployment Rollback Example (Kubernetes)

```
# Check rollout history
a kubectl rollout history deployment/myapp

# Rollback to previous version
a kubectl rollout undo deployment/myapp

# Verify
a kubectl get pods -l app=myapp
a kubectl rollout status deployment/myapp
```

5 Deployment Flow with Rollback



6 Why Rollback Must Be Planned Before Deployment

- ✓ You need a previous known-good version available.
- ✓ Your rollback process should be tested.
- ✓ Configuration and database changes may also need a rollback plan.
- ✓ Clear steps and ownership reduce downtime during an incident.
- ✓ Without a plan, rollback can be slow, risky or even impossible.



7 Why Blindly Rerunning a Failed Pipeline Can Make Incidents Worse

- ✗ It may deploy the same broken version again.
- ✗ It can overwrite a working rollback.
- ✗ It can create more failed resources, increasing confusion.
- ✗ It does not fix the root cause.
- ✗ It can make downtime longer and investigation harder.

Key Takeaways

- ✓ Always keep previous versions of your application.
- ✓ Have a tested rollback process before you deploy.
- ✓ Detect issues quickly using monitoring and alerts.
- ✓ Rollback is not a failure, it is a recovery strategy.
- ✓ Do not just rerun a failed pipeline. Find and fix the root cause.

Real-World Example Scenario

- A new version (v3) is deployed to production.
- Error rate increases and users report issues.
- The team stops the rollout.
- They rollback to the previous version (v2).
- Health checks and metrics return to normal.
- The team investigates the root cause before the next release.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Reading a Real CI/CD Pipeline

Let's look at a simple but real CI/CD pipeline configuration using GitHub Actions. This example shows a basic flow: build, test and deploy. We will break it down and connect it to the concepts from Pages 1 to 18.

1 CI/CD Pipeline Example (GitHub Actions YAML)

```
1 name: CI-CD Pipeline
2
3 on:
4   push:
5     branches: [ main ]
6
7 env:
8   APP_NAME: myapp
9   IMAGE_TAG: ${ github.sha }
10
11 jobs:
12   build:
13     runs-on: ubuntu-latest
14     steps:
15       - name: Checkout code
16         uses: actions/checkout@v4
17       - name: Set up Python
18         uses: actions/setup-python@v5
19         with:
20           python-version: '3.11'
21       - name: Install dependencies
22         run: pip install -r requirements.txt
23       - name: Build application
24         run: echo "Building $APP_NAME"
25
26   test:
27     runs-on: ubuntu-latest
28     needs: build
29     steps:
30       - name: Run tests
31         run: pytest
32
33   deploy:
34     runs-on: ubuntu-latest
35     needs: test
36     if: github.ref == 'refs/heads/main'
37     steps:
38       - name: Deploy to production
39         run: echo "Deploying $APP_NAME"
```

2 Trigger



- on: push
- The pipeline runs when code is pushed to the main branch.

3 Environment Variables



- env:
- APP_NAME: myapp
- IMAGE_TAG: \${ github.sha }
- Used across all jobs.

4 Jobs



- build, test and deploy are separate jobs.
- Jobs run in order using needs.
- Each job runs on a GitHub-hosted runner.

5 Runner



- runs-on: ubuntu-latest
- Uses a clean virtual machine provided by GitHub.
- Each job can run on a different runner.

6 Steps



- A job contains multiple steps.
- Each step runs a command or uses an action (e.g. checkout code, install dependencies).
- Steps run from top to bottom.

10 Pipeline Flow



7 Build



- Checkout code
- Install dependencies
- Build the application

8 Test



- Run automated tests
- Verify the application works

9 Deploy



- Deploy to the target environment
- Runs only if tests pass and on the main branch

11 Trace and Connect to the Architecture (Pages 1 - 18)



- Start from the trigger and read the pipeline from top to bottom.
- Identify each stage (build, test, deploy).
- See how it matches the CI/CD pipeline architecture: Code → Build → Test → Deploy.
- Understand how each part (jobs, runner, steps, environment variables) fits into the real pipeline you learned earlier.

12 Key Takeaways



- ✓ A CI/CD pipeline is defined in a YAML file.
- ✓ It has a trigger, jobs, steps and environment variables.
- ✓ Each job runs on a runner and performs specific tasks.
- ✓ The pipeline runs from top to bottom.
- ✓ This configuration connects directly to the CI/CD architecture and concepts from Pages 1 to 18.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Your First CI/CD Pipeline, Junior Engineer Lab

1 Create a small repository

- Create a new repository on GitHub.
- Initialize a simple application (e.g. Python).
- Add a README and sample code.



```
my-app/
├── app.py
├── test_app.py
└── README.md
```

2 Make a code change

- Edit the code (e.g. add a new feature or fix a bug).
- Commit and push the change.

```
1 def add(a, b):
2     return a + b
3
4 print(add(2, 3))
```

3 Trigger CI

- Push to the repository or open a pull request.
- This triggers the CI pipeline automatically.



git push

4 Run an automated test

- Pipeline installs dependencies and runs tests.
- Tests check if your code works.

```
$ pytest
✓ test_add()
✓ 1 passed in 0.21s
```

5 Build an artifact or container image

- Build the application package or Docker image.
- This becomes the artifact for deployment.



docker build -t my-app:v1.

6 Intentionally break a test

- Make a change that causes a test to fail.
- This helps you understand how failures look.

```
$ pytest
✗ test_add()
E assert 5 == 4
```

7 Read the failure logs

- Check the pipeline logs.
- Identify the exact error message.
- Understand why the test failed.

```
test_add
E assert 5 == 4
> + where 5 = add(2, 3)
```

8 Fix the problem

- Update the code to fix the issue.
- Commit and push the fix.

```
def add(a, b):
    return a + b
```

9 Rerun the pipeline

- Push the fix.
- The pipeline runs again automatically.



git push

10 Verify success

- All stages should pass.
- Confirm the artifact is built and ready for deployment.

```
✓ Build
✓ Test
✓ Deploy
✓ Pipeline completed successfully
```

Final Mental Model



Developer
Writes code



Git Push / Pull Request
Triggers the pipeline



Pipeline Trigger
Starts the CI/CD process



Runner
Executes the jobs



Checkout
Pulls the code



Build
Builds the application



Test
Runs automated tests



Quality + Security Gates
Runs linting, scanning, checks



Package Artifact
Creates deployable artifact / Docker image



Registry
Stores the artifact (e.g. Docker Registry)



Deploy
Deploys to target environment



Verify
Runs health checks and smoke tests



Monitor
Tracks logs, metrics and alerts



Production
Application is running for users



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta